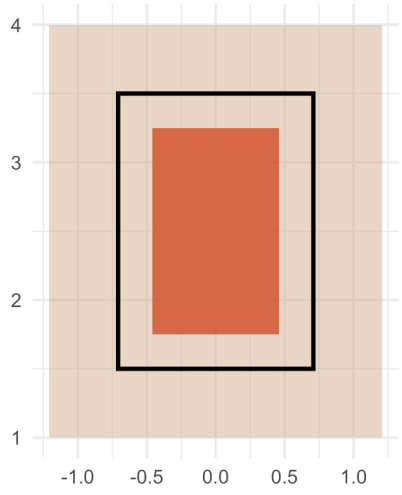
Catcher Framing
Ian Chang

Data:

For this analysis project, we were given a dataset of over 1 million rows and 36 features. After looking at each feature's distributions and data types, I cleaned the data by removing all rows with NA's in *TOP_ZONE, BOT_ZONE,* and *EXTENSION*. I also noticed pitches in our data with over 3 strikes, and pitches with vertical break over 1000 so I removed those pitches to clear outliers and potential errors.

Method:

To create a metric that determines the amount of strikes added or subtracted through their framing skill, we would have to compare the ball pitch location in contrast to the strike zone. If the pitch was called a strike but did not pass through the strike zone as it crosses the front of the plate, we count that as an '***added strike***', as we assume the catcher made a frame attempt which impacted the umpire into making a strike call. If the pitch was called a ball but did in fact pass through the strike zone, we count that as an '***subtracted strike***', as the catcher did not make a positive impact towards the call. We do not take into account umpire performance for now. To compare the umpire call to a true strike/ball calculation, we must first establish our calculated strike zone (CSZ) to calculate whether or not the pitch was a ball or strike.

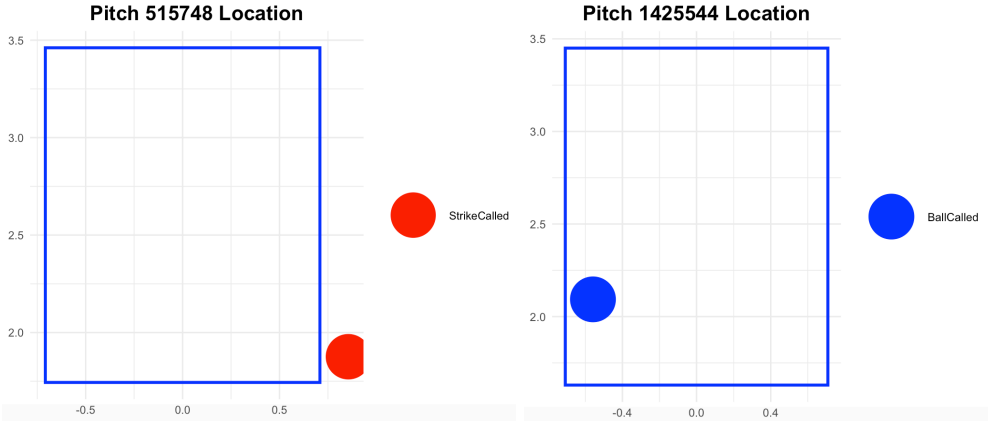


To establish our CSZ, I used the given *TOP_ZONE* and *BOT_ZONE* metrics, as well as the width of home plate for the left/right edges of the CSZ. The width of home plate is 17in, and if we establish $x = 0$ (ft) as the center of home plate, then the edges of our calculated strike zone (CSZ) go from *-0.7083ft* to *0.7083ft*. Assuming our ball location metric is the center of the ball, we add an additional *0.125ft* (roughly the radius of a baseball) to account for any part of the ball that potentially passes through the zone, making the edges of the CSZ go from -*0.833ft* to *0.833ft*.

Next, I filtered all pitches to pitches within the shadow zone (pitches within length of a ball ~3 inches of the CSZ). Not all pitches are frame worthy, such as pitches over the heart of the plate or pitches way outside. Any pitch within 3in inside of the top/bottom/left/right borders would count as frame worthy, as well as any pitches 3 inches outside of the CSZ borders. However I chose to add on another 3 inches to the outside shadow zone just to catch more framing opportunities where the edge of a ball could catch the edge of the shadow zone.

Now that we've filtered to all border pitches with optimal framing opportunities, we can calculate whether each pitch was an added strike or not. If the pitch intersected with any part of our CSZ, we would label that as a calculated strike. Then turning the umpire calls and our calculated calls into binary numbers (0 is ball, 1 is strike) we can calculate strikes added/subtracted by subtracting the umpire call by our calculated call. If a supposed calculated ball was called a strike, the catcher has added a called strike. If a supposed calculated strike was called a ball, the catcher had a negative effect and subtracted a strike. If a supposed calculated strike was called a strike, or a calculated ball was called a ball, the catcher had no effect on adding strikes.

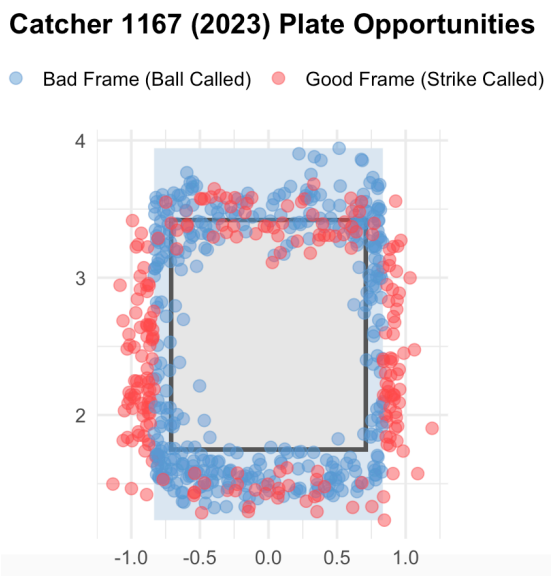
For example, here's an added strike pitch. Here's a subtracted strike pitch.

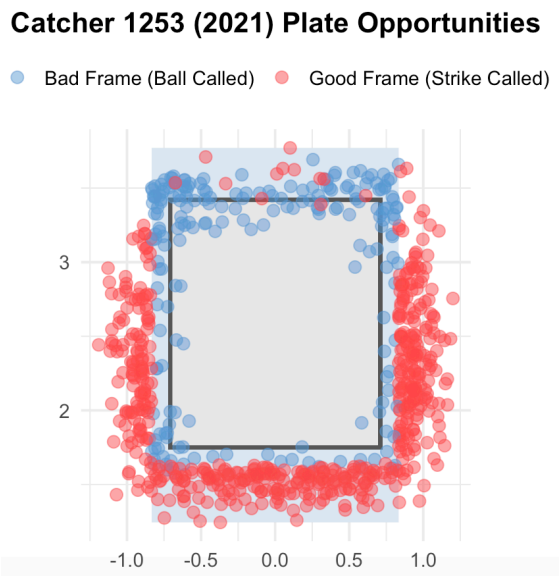


To get our new_output.csv, we group our data by *CATCHER_ID, GAME_YEAR* and total up the number of pitches caught, strikes called, and our sum of added strikes out of our framing opportunities.

Analysis:

In our final new_output.csv, catcher 1167 in 2023 had the worst strikes added with -164 strikes added. You can see in the left figure, the catcher had a bunch of missed frame opportunities where a ball was called inside of the CSZ. Whereas catcher 1253 in 2021 had the best strikes added with 357 strikes added. You can see in the right figure he converted many framing opportunities where balls outside of the zone were called strikes.





For catchers with over 300 plate opportunities, here are the top 5 catchers in added strikes/100:

	CATCHER_ID	GAME_YEAR	OPPORTUNITIES	ACTUAL_CALLED_STRIKES	CALLED_STRIKES_ADDED	CALLED_STRIKES_ADDED_PER_100
1	473	2021	361	125	23	6.3711911
2	847	2021	301	93	17	5.6478405
3	2087	2021	2046	666	102	4.9853372
4	1487	2021	709	237	35	4.9365303
5	1363	2021	571	185	28	4.9036778

Here are the bottom 5 catchers in added strikes/100:

	CATCHER_ID	GAME_YEAR	OPPORTUNITIES	ACTUAL_CALLED_STRIKES	CALLED_STRIKES_ADDED	CALLED_STRIKES_ADDED_PER_100
1	2169	2023	334	105	-11	-3.29341317
2	588	2023	3032	929	-90	-2.96833773
3	1793	2023	372	126	-10	-2.68817204
4	316	2023	979	275	-25	-2.55362615
5	1547	2022	1061	305	-27	-2.54476909

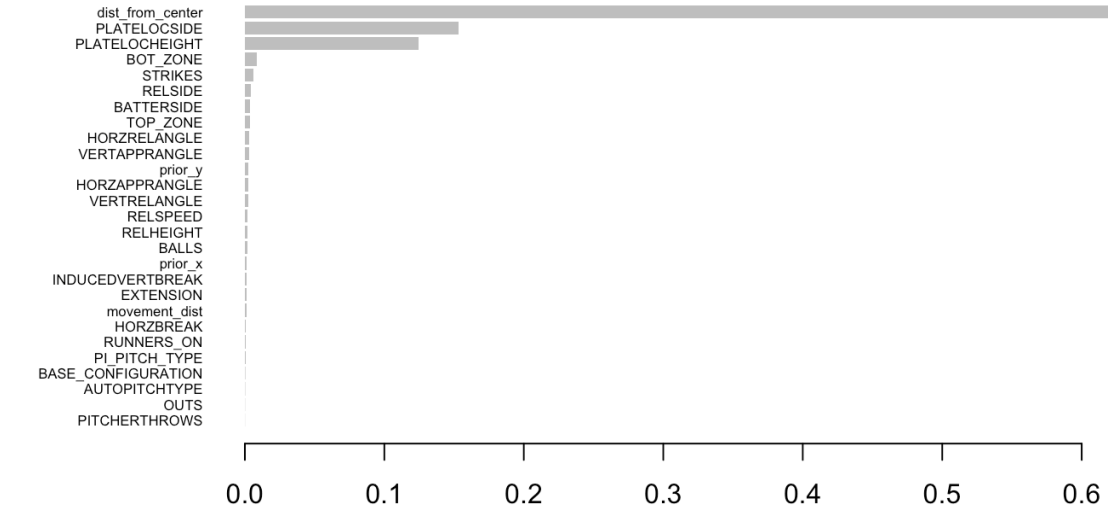
Model Method:

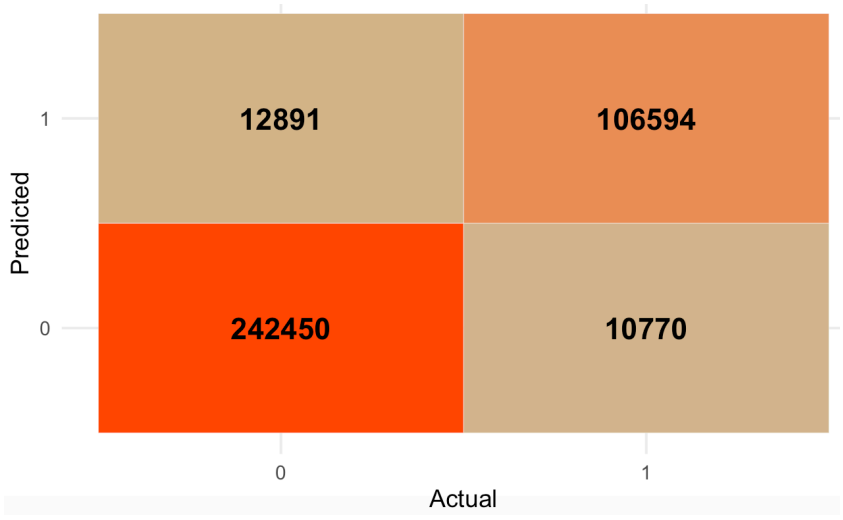
To create a model that will predict the probability of a pitch being called a strike, I took the approach of binary classification. I decided to implement my model using XGBoost, to learn which features were most important to successfully predicting the feature ***PITCHCALL***, either a strike or ball called.

I first cleaned the data to a dataset more fit for training the model, factoring classification variables into levels such as the features ***AUTOPITCHTYPE***, ***PI_PITCH_TYPE***, ***BATTERSIDE***, and ***PITCHERTHROWS***. I created binary columns to represent the ***PITCHCALL*** column, and created other metric columns to help the model increase the accuracy. I created ***DIST_FROM_CENTER*** that calculates the distance from the pitch to the center of the approximate strike zone. As well as ***prior_x***, ***prior_y*** which gives the x and y points in feet of the pitch if we assume the pitch traveled in a perfectly straight line, to help us visualize how much

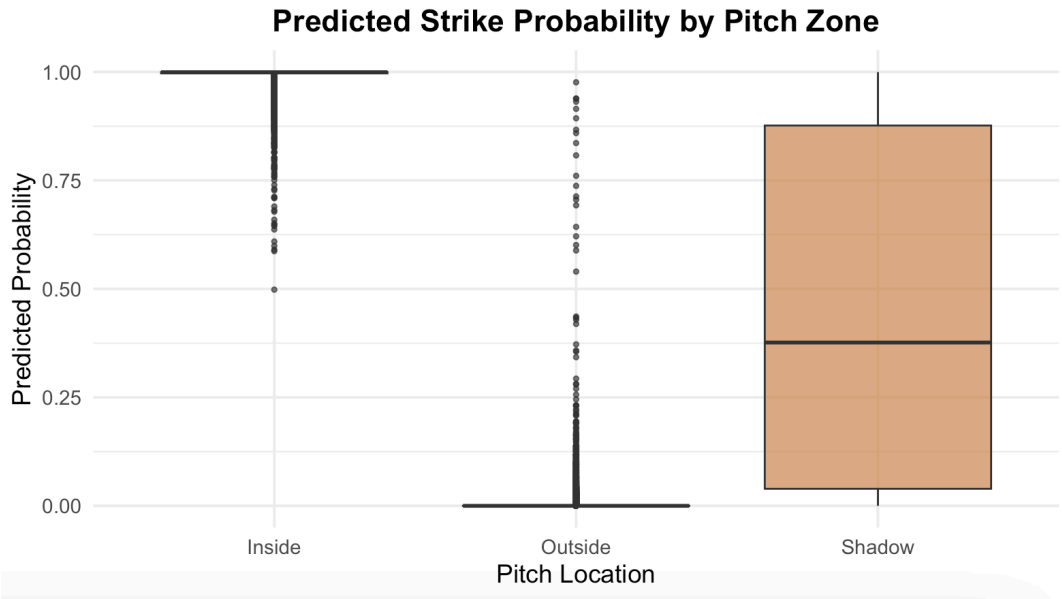
break the pitch had. Another variable ***MOVEMENT_DIST*** calculates the distance between the actual pitch and the calculated pitch location without any break. We did not take into account the game year, any of the ID columns, nor the calculated strike variable, strikes added, or pitch location variable we calculated earlier. All other variables we took into account to train our model on.

I split the data into using 2021-22 as training data and 2023 as testing data to maintain year to year predictiveness. Using XGBoost, I was able to learn weights for each variable to predict the probability of a strike being called. The plot below is the model variable importance plot for each feature used in our model, as well as the resulting confusion matrix. The distance away from the center of the strike zone is the most predictive factor in the probability of predicting a called strike. The further the pitch is from the center of the zone, the less likely it is to be called a strike. The number of strikes, batter side, and top/bottom zone estimation has little effect on the prediction, while the pitch x and y position are more of a factor in the predictiveness. In which all of these relationships make sense.

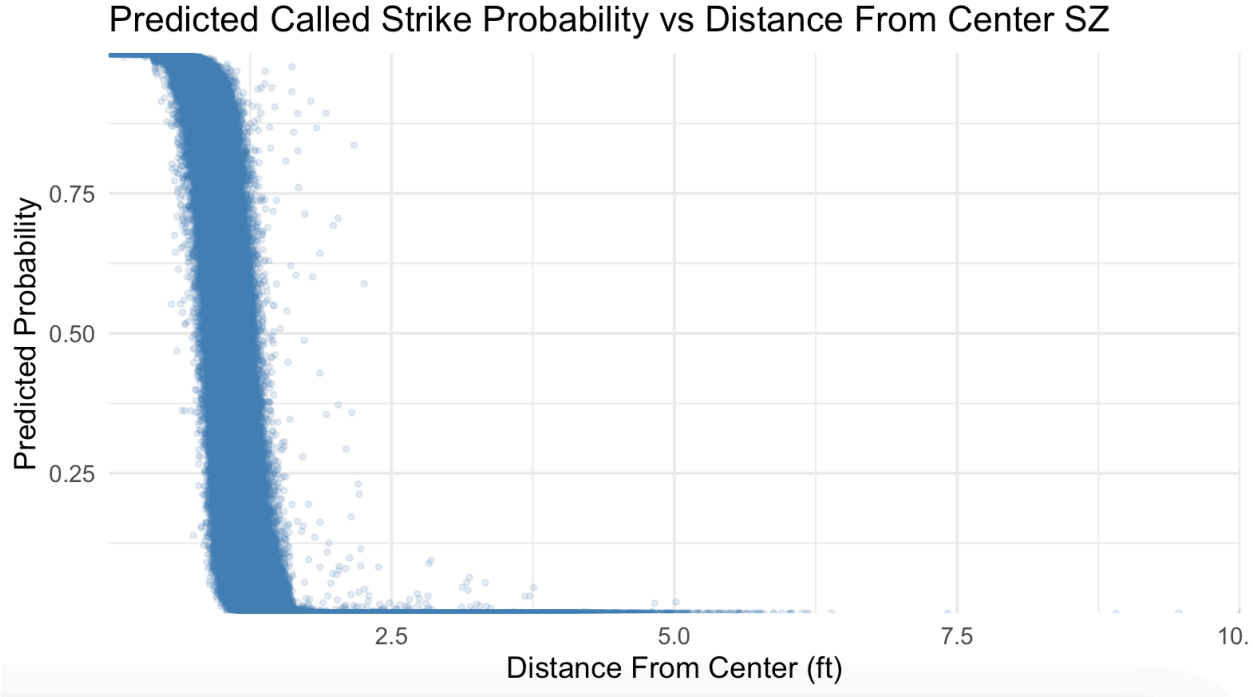




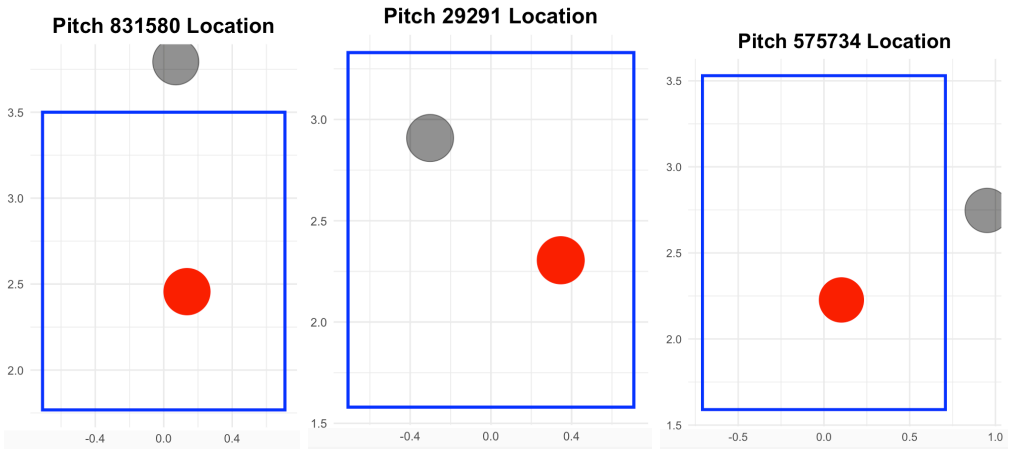
The results proved to be conclusive as the **RSME** for our model after training was **<0.213**, and had **F1** precision score of **0.9535**. Our confusion matrix shows us 106,594 correctly classified strikes, 242,450 correctly classified balls, 10,770 incorrectly classified strikes as balls, and 12,891 incorrectly classified balls as strikes, giving us a **93.65%** prediction accuracy on our test data. The average called strike probability for pitches **inside** the zone was **99.56%**, for pitches on the **border** it was **44.83%**, and **0.0898%** for pitches **outside** of the zone and border.



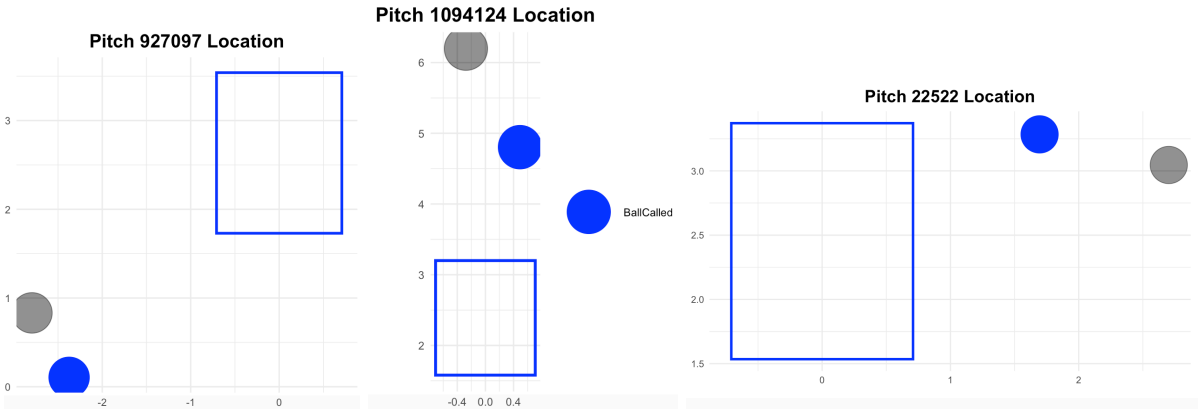
Here's the plot of the distance away from the center of the zone vs predicted called strike probability. We can see a general flow from 1 to 0 in predicted probability as the distance from the center of the zone increases.



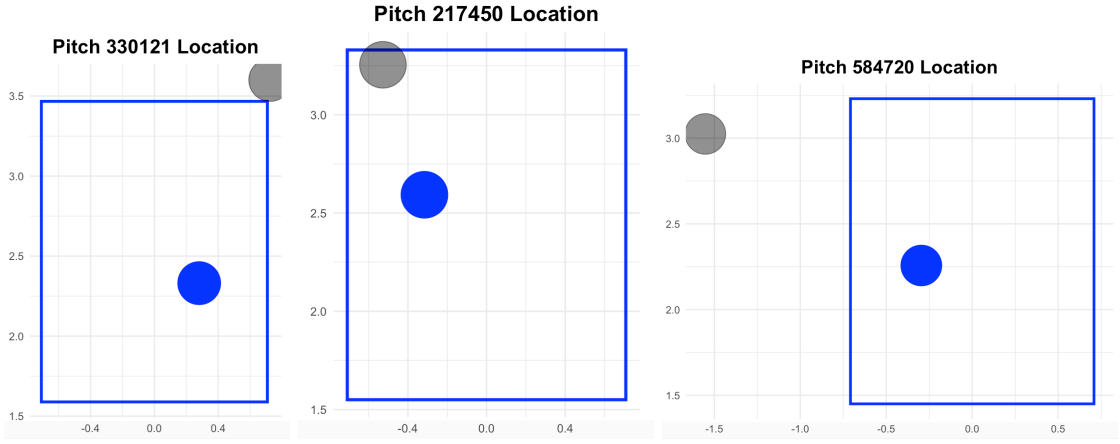
We'll look at some examples from our test data to validate our model further using an eye test. Here are some examples of pitches with high called strike probability (over 99%) that were actually called strikes:



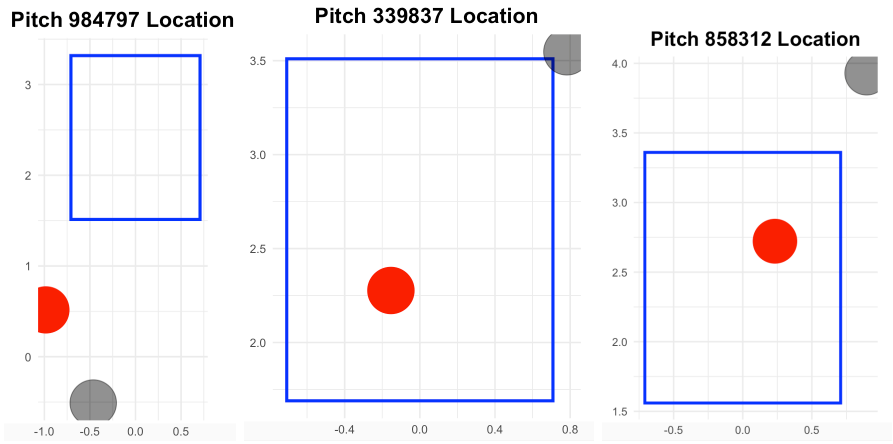
Here are some examples of pitches with low called strike probability (less than 0.00001%) that were actually called balls:



Here are some examples of pitches with high called strike probability that were actually called balls:



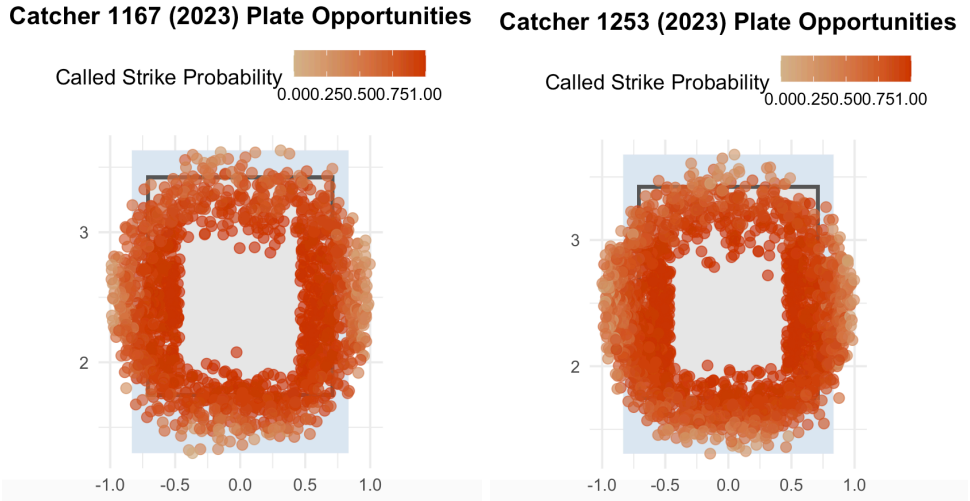
Here are some examples of pitches with low called strike probability that were actually called strikes:



Through the eye test, it seems like the high strike probability but actually called a ball in the data seems to be an umpire or data labeling error. From the eye test the pitch is well in the strike zone and should have been a strike. So our model does a good job there identifying when a pitch should actually be a strike even though it was mistakenly called a ball.

Now for pitches with low strike probability but actually called strikes, we can see one pitch for example where it seems like a labeling error. However there are a couple pitches where our model seems to be calculating them as balls but by eye test they are well in the strike zone. This is an inaccuracy in our model, and by more eye testing and my intuition it seems like the movement and break on the ball seems to be impacting how our model determines the probability for these pitches. With more time I would like to play around with the vertical and horizontal break variables.

Taking our best and worst catcher for added strikes per 100, we see a similar distribution of called strike probability around the shadow zone of the strike zone.



Given that we have missed calls, tracking errors, and potential of data input errors, our model does a pretty good job predicting the called strike probability. Especially with calls dealing with framing pitches around the shadow zone of the strike zone. With the high accuracy rates, F1 score, and low RSME, as well as validating using examples and eye tests, I feel confident in presenting and utilizing this model to predict new data.